

Praktikum 14: Keamanan API, Autentikasi Token, dan Axios Interceptors

A. Tujuan Praktikum

1. Mahasiswa mampu memahami konsep keamanan RESTful API menggunakan Token-Based Authentication.
2. Mahasiswa mampu mengimplementasikan *Filters* pada CodeIgniter 4 untuk mengamankan endpoint API dari akses ilegal.
3. Mahasiswa mampu memahami dan mengimplementasikan fungsi *Axios Interceptors* pada aplikasi Frontend VueJS.
4. Mahasiswa mampu melakukan pengujian transmisi data yang aman antara Frontend SPA dan Backend API secara end-to-end.

B. Instruksi Praktikum

1. Persiapkan text editor misalnya Visual Studio Code (VSCode).
2. Pastikan web server Apache dan MySQL pada XAMPP dalam kondisi aktif.
3. Buka folder proyek backend CodeIgniter 4 (lab7_php_ci) dan folder frontend VueJS (lab8_vuejs) yang telah dikerjakan pada Praktikum 13.
4. Ikuti langkah-langkah praktikum secara berurutan.

C. Teori Singkat

Pada praktikum sebelumnya, kita baru menerapkan *Client-Side Security* (keamanan di sisi browser menggunakan Vue Router). Keamanan tersebut belum cukup, sebab orang lain masih bisa menembak database kita secara langsung melalui URL endpoint REST API (seperti /post dengan metode POST/PUT/DELETE) menggunakan tools seperti Postman. Oleh karena itu, diperlukan *Server-Side Security* menggunakan token.

Saat pengguna berhasil login, server akan memberikan sebuah string acak unik (*Token*). Token ini wajib disimpan oleh aplikasi klien dan harus dilampirkan pada bagian HTTP Header (Authorization: Bearer <token>) setiap kali klien meminta data sensitif atau melakukan perubahan data ke server.

Di sisi klien, untuk menghindari penulisan kode pelampiran token secara manual di setiap fungsi axios.get atau axios.post, kita dapat menggunakan **Axios Interceptors**. Fitur ini berfungsi seperti "jembatan otomatis" yang mencegat setiap request keluar dari aplikasi VueJS untuk menyuntikkan token dari localStorage ke dalam HTTP Header secara global.

D. Langkah-langkah Praktikum

TAHAP 1: MENGAMANKAN ENDPOINT API (SISI BACKEND CI4)

Langkah 1.1: Membuat API Auth Filter

Kita akan membuat filter khusus yang bertugas memeriksa apakah request yang masuk ke server membawa Token valid pada HTTP Header-nya. Buat file filter baru bernama `ApiAuthFilter.php` di dalam folder `app/Filters/` dan ketikkan kode berikut:

PHP

```
<?php

namespace App\Filters;

use CodeIgniter\Filters\FilterInterface;
use CodeIgniter\HTTP\RequestInterface;
use CodeIgniter\HTTP\ResponseInterface;
use Config\Services;

class ApiAuthFilter implements FilterInterface
{
    public function before(RequestInterface $request, $arguments = null)
    {
        // 1. Ambil data Header Authorization dari request klien
        $authHeader = $request->getServer('HTTP_AUTHORIZATION');

        if (!$authHeader) {
            // Jika header tidak ditemukan, kirim respon error 401
            $response = Services::response();
            $response->setStatusCode(401);
            return $response->setJSON([
                'status' => 401,
                'error' => 401,
                'messages' => 'Akses Ditolak. Token tidak ditemukan pada request!'
            ]);
        }

        // 2. Ekstrak string token (Memisahkan kata 'Bearer' dengan string token)
        $token = null;
        if (preg_match('/Bearer\s(\S+)/', $authHeader, $matches)) {
            $token = $matches[1];
        }

        // 3. Validasi Token (Pada praktikum ini disesuaikan dengan format enkripsi di Controller Auth)
        // Catatan: Pada rilis production, bagian ini biasanya divalidasi menggunakan library JWT (JSON Web Token)
        if (!$token || empty($token)) {
            $response = Services::response();
        }
    }
}
```

```
$response->setStatusCode(401);
return $response->setJSON([
    'status' => 401,
    'error' => 401,
    'messages' => 'Sesi Token tidak valid atau kedaluwarsa!'
]);
}
}

public function after(RequestInterface $request, ResponseInterface $response, $arguments = null)
{
    // Tidak diperlukan aksi setelah request diproses
}
}
```

Langkah 1.2: Mendaftarkan Filter API

Buka file konfigurasi filter di app/Config/Filters.php. Daftarkan class filter yang baru saja dibuat ke dalam array \$aliases:

PHP

```
public array $aliases = [
    'csrf' => \\CodeIgniter\\Filters\\CSRF::class,
    'toolbar' => \\CodeIgniter\\Filters\\DebugToolbar::class,
    'honeypot' => \\CodeIgniter\\Filters\\Honeypot::class,
    'invalidchars' => \\CodeIgniter\\Filters\\InvalidChars::class,
    'secureheaders' => \\CodeIgniter\\Filters\\SecureHeaders::class,
    // Daftarkan filter baru Anda di bawah ini
    'apiauth' => \\App\\Filters\\ApiAuthFilter::class,
];
```

Langkah 1.3: Menerapkan Filter ke Route Artikel

Buka file app/Config/Routes.php. Terapkan filter 'apiauth' khusus untuk memproteksi manipulasi data artikel (POST, PUT, DELETE) agar tidak bisa ditembak sembarangan tanpa token.

PHP

```
// Mengamankan method POST, PUT, dan DELETE untuk resource /post
$routes->post('post', 'Api\\Post::create', ['filter' => 'apiauth']);
$routes->put('post/(.segment)', 'Api\\Post::update/$1', ['filter' => 'apiauth']);
$routes->delete('post/(.segment)', 'Api\\Post::delete/$1', ['filter' => 'apiauth']);
```

TAHAP 2: IMPLEMENTASI AXIOS INTERCEPTORS (SISI VUEJS FRONTEND)**Langkah 2.1: Menambahkan Konfigurasi Interceptor Global pada assets/js/app.js**

Buka berkas utama frontend Anda di assets/js/app.js. Tambahkan konfigurasi pencegat (*Interceptor*) Axios tepat sebelum baris kode inisialisasi `const app = createApp({})`;

Fungsi ini akan otomatis menyuntikkan token yang tersimpan di `localStorage` ke dalam setiap request data menuju ke backend.

JavaScript

```
const { createApp } = Vue;
const { createRouter, createWebHashHistory } = VueRouter;

const apiUrl = 'http://localhost/labci4/public';

// =====
// IMPLEMENTASI AXIOS INTERCEPTORS (Penyuntik Token Otomatis)
// =====
axios.interceptors.request.use(
  (config) => {
    // Ambil token dari local storage browser
    const token = localStorage.getItem('userToken');

    // Jika token tersedia, masukkan ke dalam HTTP Header Authorization Bearer
    if (token) {
      config.headers['Authorization'] = 'Bearer ' + token;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Tangkap secara global jika server merespon dengan error 401 (Unauthorized)
axios.interceptors.response.use(
  (response) => {
    return response;
  },
  (error) => {
    if (error.response && error.response.status === 401) {
      alert('Sesi Anda telah berakhir atau Token tidak sah. Silakan login kembali.');
```



```
      localStorage.clear(); // Bersihkan local storage
      window.location.href = '#/login'; // Tendang paksa ke halaman login
    }
  }
);
```

```
        window.location.reload();
    }
    return Promise.reject(error);
}
);
// =====

// 1. Definisikan mapping rute URL ke Komponen
const routes = [
  { path: '/', component: Home },
  { path: '/login', component: Login },
  {
    path: '/artikel',
    component: Artikel,
    meta: { requiresAuth: true }
  }
];

const router = createRouter({
  history: createWebHashHistory(),
  routes
});

// 2. Navigation Guards (Client-Side Security)
router.beforeEach((to, from, next) => {
  const isAuthenticated = localStorage.getItem('isLoggedIn') === 'true';

  if (to.matched.some(record => record.meta.requiresAuth) && !isAuthenticated) {
    alert('Akses Ditolak! Anda harus login terlebih dahulu.');
```

```
    next('/login');
```

```
  } else {
    next();
  }
});

// 3. Inisialisasi Root Instance VueJS Application
const app = createApp({
  data() {
    return {
      isLoggedIn: false
    }
  },
```

```
mounted() {
  this.isLoggedIn = localStorage.getItem('isLoggedIn') === 'true';
},
methods: {
  logout() {
    if (confirm('Apakah Anda yakin ingin keluar aplikasi?')) {
      localStorage.clear();
      this.isLoggedIn = false;
      this.$router.push('/');
    }
  }
}
});

app.use(router);
app.mount('#app');
```

E. Pertanyaan dan Tugas

1. Terapkan seluruh langkah pembuatan kode pengamanan *Token-Based Authentication* di atas baik pada backend (CI4) maupun frontend (VueJS).
2. Lakukan simulasi pengujian pembobolan database menggunakan aplikasi REST Client seperti **Postman** atau **Insomnia**:
 - o Tembak URL input data artikel backend Anda (<http://localhost/labci4/public/post>) menggunakan metode POST tanpa menyertakan token di bagian headers.
 - o Amati hasil respon JSON yang diberikan oleh CodeIgniter 4. Pastikan statusnya mengembalikan error HTTP 401 Unauthorized.
3. Buka aplikasi web utama lewat browser, lakukan login, lalu coba lakukan manipulasi data artikel (Tambah/Ubah/Hapus). Berkat *Axios Interceptors*, proses manipulasi data harus berjalan sukses karena token dikirim secara terselubung di latar belakang sistem.
4. Tuliskan kesimpulan akhir dari hasil analisis Anda mengenai perbedaan mendasar fungsi perlindungan keamanan antara *Vue Router Navigation Guards* (sisi klien) dan *CodeIgniter Filters* (sisi server).

F. Laporan Praktikum

1. Melanjutkan repositori pengerjaan GitHub sebelumnya dengan nama Lab11Web_VueJS.
2. Ambil gambar screenshot bukti penolakan akses error 401 dari Postman saat mencoba menembak API tanpa token.
3. Ambil gambar screenshot tab *Network* pada browser Developer Tools (tekan F12 -> pilih tab Network -> klik pada request nama API) untuk memperlihatkan bukti bahwa parameter Authorization: Bearer <string_token> telah berhasil disuntikkan secara

otomatis oleh Axios Interceptors.

4. Perbarui file README.md, tuliskan rangkuman penjelasan singkat dari setiap langkah praktikum 14 beserta lampiran screenshotnya.
5. Lakukan penguncian commit akhir dan lakukan push seluruh pembaruan proyek ke GitHub. Setorkan alamat URL repositori Anda pada platform pembelajaran ecampus Universitas Pelita Bangsa.